TP # 23

Polynôme interpolateur de Lagrange

Rappels : Représenter un graphe de fonction

Bibliothèque matplotlib.pyplot

```
import matplotlib.pyplot as plt
```

Pour tracer la courbe représentative de f sur un segment $[a, b]$:

1 Pour le vecteur des abscisses : `X = np.linspace(a,b,n)`

Rappel : ceci crée un vecteur composé de n points uniformément répartis entre a et b. On choisira typiquement $n = 100$ ou 1000.

2 Pour le vecteur des ordonnées : `Y = f(X)`

Remplacer directement `f(X)` par l'expression voulue.
Exemple, pour représenter la foncction $f : x \mapsto e^{-x^2}$: `Y = np.exp(-X**2)`.

3 Relier les points avec `plt.plot(X,Y)` puis afficher la représentation avec `plt.show()`.

Exercice 1

La théorie.

Soit $n \in \mathbb{N}$ et $x_0 < x_1 < \ldots < x_n$ des réels distincts.
On définit les polynômes :

$$\forall i \in [\![0, n]\!], \quad L_i(X) = \prod_{j \in [\![0,n]\!] \setminus \{i\}} \frac{X - x_j}{x_i - x_j}$$

1. Pour tous $i, k \in [\![0, n]\!]$, compléter les valeurs suivantes :

$$\text{Si } k \neq i, \quad L_i(x_k) = \ldots.. \qquad \text{Si } k = i, \quad L_i(x_i) = \ldots..$$

2. Soit un polynôme P de la forme $P(X) = \sum_{k=0}^{n} \lambda_k L_k(X)$ avec $\lambda_0, \ldots, \lambda_n \in \mathbb{R}$.

Compléter les valeurs suivantes :

$$P(x_0) = \ldots..\ , \quad P(x_1) = \ldots..\ , \quad \ldots\ , \quad P(x_n) = \ldots..$$

3. Compléter la preuve que $\mathcal{B} = (L_0, L_1, \ldots, L_n)$ est une base de $\mathbb{R}_n[X]$:

- Puisque $Card(\mathcal{B}) = \ldots\ldots\ldots\ldots$ et $\dim(\mathbb{R}_n[X]) = \ldots\ldots\ldots\ldots$,

il suffit de montrer que .....

- Soient $\lambda_0, \ldots, \lambda_n \in \mathbb{R}$. On suppose que $\sum_{k=0}^{n} \lambda_k L_k(X) = 0$ (égalité dans $\mathbb{R}[X]$).

- En évaluant en ..... on obtient : .....

- En évaluant en ..... on obtient : .....

etc

- En évaluant en ..... on obtient : .....

On a montré que $\lambda_0 = \lambda_1 = \ldots = \lambda_n = 0$, CQFD.

4. Ainsi, tout $P \in \mathbb{R}_n[X]$ se décompose (de manière unique) dans la base $\mathcal{B}$.
Préciser quelle est cette décomposition :

$$P(X) = \sum_{k=0}^{n} \ldots\ldots\ldots\ldots\ L_k(X)$$

5. On en déduit que si $x_0 < x_1 < \ldots < x_n$ et si $y_0, y_1, \ldots, y_n \in \mathbb{R}$, il existe un unique polynôme P de degré $\leqslant n$ satisfaisant : $\forall k \in [\![0, n]\!],\ P(x_k) = y_k$.
Précisément, on peut donner la forme de ce polynôme : il s'agit de

$$P(X) = \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots$$

On l'appelle le **polynôme interpolateur de Lagrange** pour les points (x_0, y_0), (x_1, y_2), $\ldots (x_n, y_n)$.

Exercice 2

La pratique.

1. Compléter la définition de la fonction `poly_L` qui prend en entrée :

- Une liste `x = [` $x_0, x_1, \ldots, x_n$ `]` de réels deux à deux distincts,
- Un entier `i`$\in [\![0, n]\!]$,
- Un réel `t`,

et renvoie la valeur du réel $L_i(t) = \displaystyle\prod_{j \in [\![0,n]\!] \setminus \{i\}} \frac{t - x_j}{x_i - x_j}$

Rappel : `len(A)` permet d'accéder au nombre d'éléments d'une liste `A`.

```
def poly_L(x,i,t) :

      n = ................

      L = ...

      for j in range( ..... ) :

        if i != j :

              L = L * ................................

      return L
```

2. Compléter la définition de la fonction `poly_P` qui prend en entrée :

- Une liste `x = [` $x_0, x_1, \ldots, x_n$ `]` de réels deux à deux distincts,
- Une liste `y = [` $y_0, y_1, \ldots, y_n$ `]` de réels,
- Un réel `t`,

et renvoie la valeur de $P(t)$,
où P est le polynôme interpolateur de Lagrange défini précédemment.

```
def poly_P(x,y,t) :

     n = .............

     S = ...

     for k in range( ..... ) :

          S = S + ....................................

     return S
```

3. Compléter et taper le programme suivant pour afficher le graphe du polynôme interpolateur de Lagrange (avec les points spécifiés) sur l'intervalle $[0, 5]$.

```
impot numpy as np
import matplotlib.pyplot as plt

x = [0, 1, 2, 5] ; y = [2, -1, 6, 3]

def P(t) :
        return poly_P(x,y,t)


X = ........................

Y = ........................

plt.plot(X,Y) # pour afficher le graphe de P
plt.plot(x, y,'rx') # pour afficher les points avec des croix rouges
plt.show()
```

On pourra modifier les listes de points à interpoler, en ajouter / en retirer, et constater les modifications sur le polynôme affiché.